

Etap 3 i Etap 4

MusicaBI – Weronika Kaźmierczak, Paola Jaroń, Mikołaj Lepsy, Adam Basiewicz

Finalny plan procesu

Na podstawie przygotowanego w poprzednim etapie wstępnego planu projektu zdefiniujcie:

1. Jakie konkretnie macie problemy z surowymi danymi? (np. braki w adresach, różne formaty dat, literówki w nazwach klientów) i jak zamierzacie je naprawić? (np. usuwanie spacji, ujednolicanie wielkości liter, pobieranie brakujących danych z innego systemu).

Anomalia	Źródło	Naprawa
Różna wielkość liter i białe znaki w nazwach artystów (np. „taylor swift” vs „Taylor Swift ”)	Last.fm, MusicBrainz, Spotify	Klucz join-owy normalizowany przez <code>lower(trim(name))</code> . Atrybut wyświetlany pozostaje w oryginalnej formie z Last.fm.
Duplikaty artystów po wielu uruchomieniach pipeline'u (Last.fm: jeden artysta x wiele scraped_date)	Last.fm	<code>DISTINCT ON (lower(trim(artist_name))) ORDER BY scraped_at DESC</code> — zachowanie najnowszego rekordu.
Brak musicbrainz_id (mbid) w odpowiedzi Last.fm dla części artystów (pusty string)	Last.fm	<code>NULLIF(mbid, '') + COALESCE</code> z mbid pobranym z MusicBrainz.
Niejednorodny format release_date (YYYY, YYYY-MM, YYYY-MM-DD)	Spotify	<code>CASE release_date_precision</code> : dla 'year' dopisanie '-01-01', dla 'month' dopisanie '-01', dla 'day' rzutowanie wprost do DATE.
Wartości w kolumnie country, które nie są kodami ISO 3166-1 (np. nazwy obszarów geograficznych)	MusicBrainz	Walidacja regex <code>^[A-Z]{2}\$</code> → niepasujące zastępowane wartością 'Unknown'.
„Szum” w tagach Last.fm — wartości typu sad, seen live, 90s mylnie traktowane jako gatunki	Last.fm	Filtr <code>tag_count >= 30</code> w <code>dim_genre</code> . Pełna wersja pozostaje dostępna w <code>dim_community_tag</code> .

2. Dla każdej tabeli wymiarów (np. Klienci, Produkty, Sklepy) oraz każdej tabeli faktów (np. Sprzedaż, Zamówienia) musicie przygotować metryczkę

Metryczka: DIM_ARTIST

Centralna tabela monitorowanych artystów — kontekst dla wszystkich metryk popularności.

Aspekt	Opis
Źródło	staging.stg_lastfm_artists (kręgosłup – Last.fm), staging.stg_musicbrainz_artists (wzbogacenie: country, debut_year, mbid), staging.stg_spotify_albums (spotify_artist_id z denormalizacji)
Transformacje	• Bazujemy na liście artystów z Last.fm — dla każdej nazwy zachowujemy tylko jeden, najświeższy rekord (nazwy porównujemy bez względu na wielkość liter i białe znaki, żeby „Taylor Swift” i „taylor swift ” trafiły do jednego wiersza). • Każdemu artyście nadajemy kolejny numer porządkowy jako artist_id (klucz sztuczny). • Dane wzbogacamy: doklejamy kraj i rok debiutu z MusicBrainz oraz identyfikator Spotify — dopasowanie po znormalizowanej nazwie artysty. • Pole country walidujemy — zostawiamy tylko poprawne dwuliterowe kody ISO (np. „US”, „PL”); resztę zastępujemy etykietą „Unknown”.
Kontrola jakości	• Brak nazwy artysty → rekord pominięty (NOT NULL) • Niepoprawny country → 'Unknown' (substitucja domyślną wartością) • Brak mbid w obu źródłach → pole zostaje NULL (akceptowalne) • Brak debut_year → NULL (akceptowalne, dane biograficzne nieobowiązkowe)
Weryfikacja (testy)	• Test dbt: artist_id unique + not_null • Test dbt: name not_null • Test manualny: SELECT COUNT(*) FROM dim_artist GROUP BY lower(trim(name)) HAVING COUNT(*)>1 — musi zwrócić 0 rekordów • Test manualny: SELECT COUNT(*) FROM dim_artist WHERE country NOT IN (SELECT DISTINCT country FROM dim_artist) — sprawdzenie spójności słownika krajów

Metryczka: DIM_DATE

Statyczny kalendarz pokrywający pełny okres scrapingu z pełną hierarchią czasową.

Aspekt	Opis
Źródło	staging.stg_lastfm_artists.scraped_date (jedynie do wyznaczenia granicy dolnej zakresu) + CURRENT_DATE (granica górna)
Transformacje	• Generujemy ciąg wszystkich dni od pierwszego dnia pobierania danych aż do dziś (po jednym wierszu na dzień). • Każdemu dniowi nadajemy kolejny numer porządkowy jako date_id. • Wyciągamy z daty atrybuty hierarchii: dzień, tydzień, miesiąc, kwartał, rok — żeby umożliwić drążenie danych na każdym poziomie.
Kontrola jakości	• Brak luki w sekwencji dat (gwarantowane przez generate_series) • Wszystkie atrybuty hierarchii NOT NULL (gwarantowane przez funkcje EXTRACT) • Materializacja jako TABLE — dane fizycznie zapisane, niezależne od trafień cache
Weryfikacja (testy)	• Test dbt: date_id unique + not_null • Test dbt: full_date unique + not_null • Test manualny: SELECT MAX(full_date) - MIN(full_date) FROM dim_date — porównać z liczbą rekordów (musi być równe COUNT-1)

Metryczka: DIM_ALBUM

Katalog wydawnictw z parsowanymi datami premier.

Aspekt	Opis
Źródło	staging.stg_spotify_albums (216 albumów dla 100 artystów)
Transformacje	• Ujednolicamy release_date — Spotify zwraca daty w trzech różnych precyzjach: pełna („2024-03-15”), sam miesiąc („2024-03”) lub sam rok („2024”). Brakujące elementy uzupełniamy pierwszym dniem miesiąca / pierwszym stycznia, żeby wszystkie wartości miały typ DATE. • Każdemu albumowi nadajemy kolejny numer porządkowy jako album_id. • Dopinamy klucz obcy do dim_artist po znormalizowanej nazwie artysty.
Kontrola jakości	• Brak spotify_album_id → rekord pominięty (WHERE spotify_album_id IS NOT NULL) • Brak release_date → musi być wypełnione (gwarantowane przez Spotify API) • Brak dopasowania artysty w dim_artist → artist_id = NULL (akceptowalne; album „osierocony” nie wystąpi w joinach faktu)
Weryfikacja (testy)	• Test dbt: album_id unique + not_null • Test dbt: spotify_album_id unique + not_null • Test manualny: SELECT COUNT(*) FROM dim_album WHERE release_date > CURRENT_DATE — musi zwrócić 0 (brak przyszłych dat) • Test manualny: SELECT COUNT(*) FROM dim_album WHERE artist_id IS NULL — wykryte „osierocone” rekordy

Metryczka: DIM_GENRE

Słownik kuratorowanych gatunków muzycznych — filtr na wadze tagu eliminuje szum.

Aspekt	Opis
Źródło	staging.stg_lastfm_artist_tags (z filtrem tag_count >= 30)
Transformacje	• Z surowych tagów Last.fm wybieramy tylko te o wadze co najmniej 30 — w ten sposób odsiewamy szum typu „sad”, „seen live” czy „90s” i zostawiamy rzeczywiste gatunki. • Nazwy ujednolicamy (małe litery, bez zbędnych spacji), żeby „Rock” i „rock” stały się jednym gatunkiem. • Każdemu gatunkowi nadajemy kolejny numer porządkowy jako genre_id.
Kontrola jakości	• Próg tag_count >= 30 dobrany empirycznie — odsiewa „sad”, „seen live”, „90s”, „female vocalists” itd. • Brak duplikatów po lower(trim) (DISTINCT na poziomie sub-query) • Brak null-i w nazwie gatunku
Weryfikacja (testy)	• Test dbt: genre_id unique + not_null • Test dbt: genre_name unique + not_null • Test manualny przeglądowy: SELECT genre_name FROM dim_genre ORDER BY genre_name — sprawdzenie, czy nie ma jawnie niepoprawnych pozycji

Metryczka: DIM_COMMUNITY_TAG

Pełna folksonomia Last.fm — wszystkie tagi bez filtra (wsparcie dla niszowych analiz).

Aspekt	Opis
Źródło	staging.stg_lastfm_artist_tags
Transformacje	• Te same kroki standaryzacji co dla gatunków (małe litery, bez spacji), ale bez filtra wagi — zachowujemy wszystkie tagi, łącznie z niszowymi. • Usuwamy duplikaty. • Każdemu tagowi nadajemy kolejny numer porządkowy jako tag_id.
Kontrola jakości	• Tylko warunek: tag_name IS NOT NULL • Brak duplikatów po lower(trim)
Weryfikacja (testy)	• Test dbt: tag_id unique + not_null • Test dbt: tag_name unique + not_null

Metryczki: BRIDGE_ARTIST_GENRE i BRIDGE_ARTIST_TAG

Tabele mostkowe realizujące relację artysta <-> gatunek/tag.

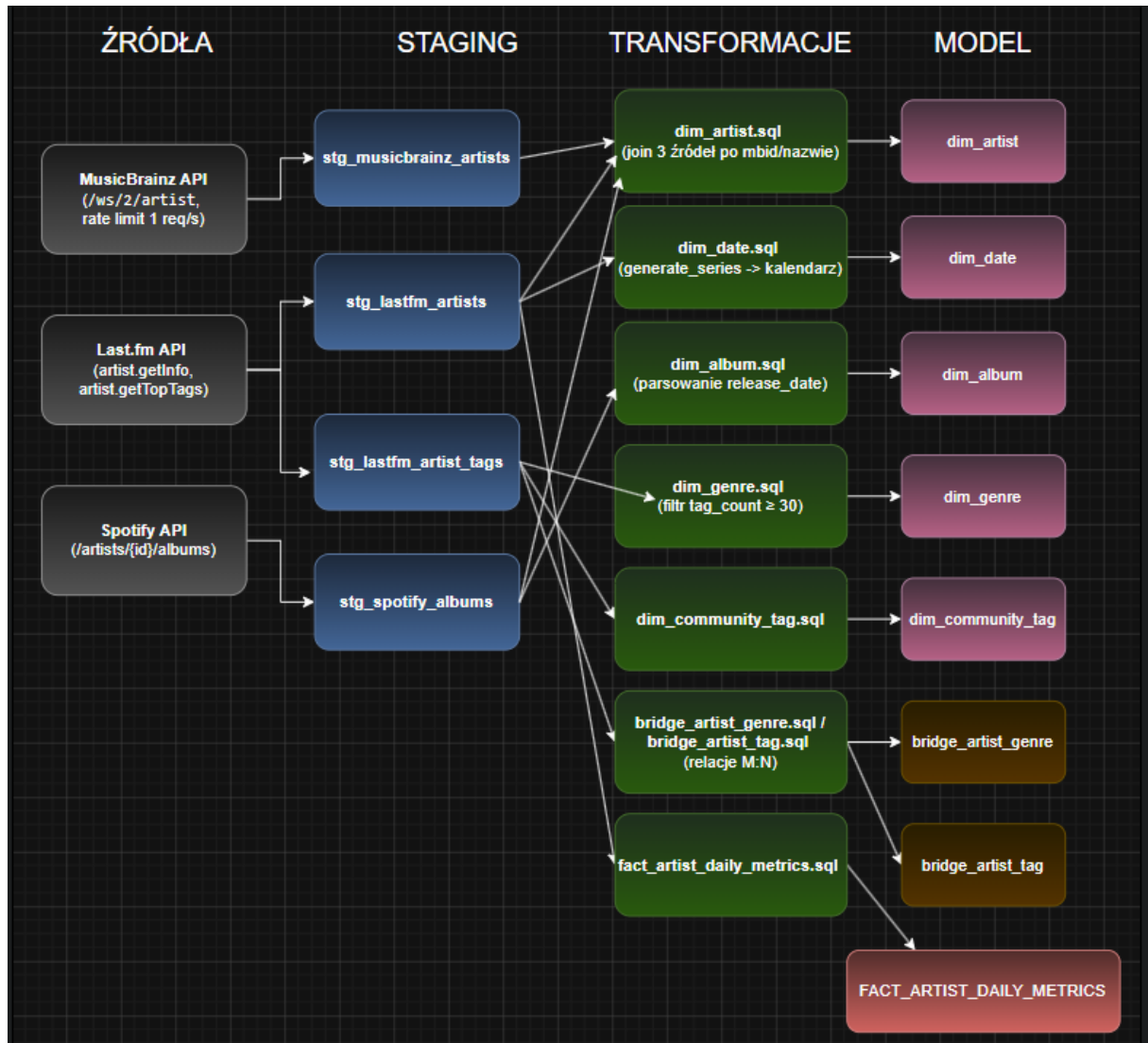
Aspekt	Opis
Źródło	staging.stg_lastfm_artist_tags + dim_artist + (dim_genre dim_community_tag)
Transformacje	• Łączymy każdego artystę z jego gatunkami / tagami — to relacja typu „wielu do wielu”, więc trzymamy ją w osobnej tabeli mostkowej. • Zachowujemy tag_count jako wagę powiązania — dzięki temu można powiedzieć, który gatunek jest dla artysty „główny” (ten z najwyższą wagą). • W bridge_artist_genre stosujemy ten sam próg co w dim_genre (waga ≥ 30), żeby tabela mostkowa była spójna z kuratorowanym słownikiem gatunków.
Kontrola jakości	• Tylko prawidłowe powiązania (INNER JOIN — brak osieroconych krawędzi) • Klucze obce wskazują wyłącznie na istniejące artist_id, genre_id, tag_id
Weryfikacja (testy)	• Test manualny: SELECT COUNT(*) FROM bridge_artist_genre WHERE artist_id NOT IN (SELECT artist_id FROM dim_artist) — musi zwrócić 0.

Metryczka: FACT_ARTIST_DAILY_METRICS

Centralna tabela faktów — codzienne metryki popularności na ziarnie (artysta, dzień).

Aspekt	Opis
Źródło	staging.stg_lastfm_artists (codzienne snapshoty listeners + playcount)
Transformacje	• Wyciągamy z Last.fm codzienne migawki: nazwa artysty, data, liczba słuchaczy, liczba odtworzeń. • Liczymy dynamikę wzrostu (growth_velocity) — czyli procentową zmianę liczby słuchaczy w stosunku do dnia poprzedniego dla tego samego artysty. • Dopinamy klucze obce — zamieniamy nazwę artysty na artist_id, a datę na date_id. • Liczymy breakout_score (skala 0–100) jako kombinację dwóch rankingów w obrębie dnia: 70% wagi to pozycja artysty w rankingu dynamiki wzrostu (im szybciej rośnie, tym wyżej), a 30% to „ premia za niszowość ” (im mniej słuchaczy, tym wyżej).
Kontrola jakości	• Brak ujemnych listeners/playcount (gwarancja Last.fm API — wartości monotonicznie rosnące) • growth_velocity NULL dla pierwszego dnia każdego artysty (oczekiwane — LAG zwraca NULL) • Dzielenie przez 0 zabezpieczone przez NULLIF(LAG(...), 0) • Wszystkie wymagane FK (artist_id, date_id) NOT NULL — wymuszone przez INNER JOIN
Weryfikacja (testy)	• Test dbt: artist_id not_null • Test dbt: date_id not_null • Test manualny: SELECT COUNT(*) FROM fact_artist_daily_metrics WHERE artist_id NOT IN (SELECT artist_id FROM dim_artist) — referencja integralności = 0 • Test manualny: SELECT COUNT(*) FROM fact_artist_daily_metrics WHERE lastfm_listeners < 0 OR lastfm_playcount < 0 — wartości graniczne = 0 • Test manualny: SELECT COUNT(*) FROM fact_artist_daily_metrics WHERE breakout_score NOT BETWEEN 0 AND 100 — zakres miary = 0

3. Oprócz opisu tekstowego macie narysować diagram, który finalnie pokaże przepływ: od źródeł, przez warstwę czyszczenia aż do gotowych tabel Wymiarów i Faktów.



Implementacja ETL

1. Pobieranie danych (Ekstrakcja) :

Tryby dostępu do danych źródłowych

Źródło	Tryb dostępu	Autentykacja	Strategia obsługi błędów
Last.fm	REST API (HTTP GET)	API key (dlt.secrets)	requests.raise_for_status() — błąd HTTP propagowany do dlt, który zaznacza job jako failed.
MusicBrainz	REST API (HTTP GET)	Brak (User-Agent wymagany)	try/except + fallback mock_mbid_<name>. Globalne time.sleep(1,1) respektuje politykę 1 req/s.
Spotify	REST API + OAuth2 (client_credentials)	Client ID + Secret	Retry z exponential backoff na HTTP 429.

Wykrywanie błędów na etapie pobierania

Każda funkcja typu fetch zwraca jednoznacznie typowany obiekt (dict lub None). Brak danych jest rozróżniany od błędu HTTP:

- Sukces (HTTP 200 + dane) → dict z wynikiem.
- Sukces (HTTP 200, ale pusta odpowiedź — np. nieznany artysta) → None lub pusta lista (rekord nie trafia do staging).
- Błąd HTTP → wyjątek (raise_for_status); dlt loguje go jako failed_job w danym pakiecie ładowania, a pozostałe rekordy są zachowywane.
- Limit API (429) → automatyczny retry (Spotify) lub graceful degradation (pozostałe).

Funkcja pomocnicza _log() w etl iteruje po load_packages i raportuje wszystkie failed_jobs w czytelnej formie — pozwala to operatorowi pipeline'u natychmiast zlokalizować problem bez konieczności wertowania logów Airflowa.

Ekstrakcja przyrostowa

Tabela staging	Write disposition	Klucz scalania / efekt
stg_lastfm_artists	merge	Klucz (artist_name, scraped_date). Ten sam dzień scrapingu jest upsertem; kolejne dni dopisują rekordy.

stg_lastfm_artist_tags	replace	Tagi nie mają historii czasowej — pełne odświeżenie przy każdym uruchomieniu.
stg_musicbrainz_artists	replace	Metadane stałe (kraj, rok debiutu) — pełne odświeżenie wystarczy.
stg_spotify_albums	replace	Katalog wydawnictw — pełne odświeżenie zapewnia spójność listy bez ryzyka „osieroconych” albumów po usunięciu z katalogu artysty.

Krytyczna konsekwencja merge disposition na stg_lastfm_artists: każde dobowe uruchomienie pipeline'u dokłada (lub aktualizuje) 100 wierszy (po jednym na artystę z listy etl/artists.txt), budując tym samym szereg czasowy listeners/playcount. Po kilku dniach kolumna growth_velocity zaczyna zwracać znaczące wartości.

2. Przetwarzanie i porządkowanie danych (Transformacja)

Wszystkie operacje czyszczenia i wzbogacania realizowane są w wydzielonym schemacie staging w PostgreSQL. dlt zapewnia, że staging jest fizycznie oddzielony od warstwy konsumpcji (schemat dimensional), a wszystkie tabele posiadają dodatkowe metadane _dlt_load_id, _dlt_id pozwalające śledzić pochodzenie każdego rekordu.

Wymiar Czasu — kalendarz statyczny

Wymiar Czasu (DIM_DATE) został zaimplementowany jako tabela statyczna generowana funkcją generate_series. Generator obejmuje pełną hierarchię (Dzień → Tydzień → Miesiąc → Kwartał → Rok), a granica górna (CURRENT_DATE) gwarantuje, że tabela jest automatycznie poszerzana z każdym uruchomieniem.

Pozostałe wymiary — standaryzacja i kolumny obliczeniowe

- dim_artist: lower(trim(name)) jako klucz join-owy; CASE/regex dla walidacji country; COALESCE dla mbid (Last.fm → MusicBrainz).
- dim_album: parsowanie release_date w zależności od release_date_precision do typu DATE; ROW_NUMBER() jako surogat.
- dim_genre: filtr tag_count >= 30 (kuracja); lower(trim) na nazwie gatunku — eliminuje duplikaty „Rock” vs „rock”.
- dim_community_tag: (bez filtra) — także lower(trim) dla nazw.

Fakty i miary

Tabela fact_artist_daily_metrics jest budowana w trzech CTE (lastfm_daily → with_velocity → joined), co rozdziela trzy logiczne etapy: ekstrakcję dziennych snapshotów, wyliczenie growth_velocity przez funkcję okienkową LAG, oraz dopięcie kluczy obcych artist_id i date_id. Końcowy SELECT dodaje breakout_score liczony przez podwójne PERCENT_RANK (po growth_velocity i po listeners).

3. Ładowanie i modelowanie danych

Architektura ROLAP

Wszystkie modele dbt są materializowane jako TABLE w schemacie dimensional. Oznacza to fizyczne zapisanie wyników wierszy do bazy, co daje natychmiastową dostępność danych dla narzędzi BI bez konieczności wykonywania kosztownych join-ów przy każdym zapytaniu.

Mechanizmy analityczne — agregaty i miary obliczeniowe

- Miary nieaddytywne `growth_velocity` i `breakout_score` są zaimplementowane bezpośrednio w `fact_artist_daily_metrics`, więc nie ma konieczności replikowania logiki w warstwie BI.
- Miary dodane w PowerBI:

```
Top Country =
    MAXX(
        TOPN(
            1,
            ALL('dimensional dim_artist'[country]),
            CALCULATE(SUM('dimensional fact_artist_daily_metrics'[lastfm_listeners]))
        ),
        'dimensional dim_artist'[country]
    )

Artist Count = DISTINCTCOUNT('dimensional bridge_artist_genre'[artist_id])

Tag Weight = SUM('dimensional bridge_artist_tag'[tag_count])

Top Genre =
    MAXX(
        TOPN(
            1,
            ALL('dimensional dim_genre'),
            CALCULATE(DISTINCTCOUNT('dimensional bridge_artist_genre'[artist_id]))
        ),
        'dimensional dim_genre'[genre_name]
    )

Top Breakout Artist =
    MAXX(
        TOPN(
            1,
            ALL('dimensional dim_artist'),
            CALCULATE(MAX('dimensional fact_artist_daily_metrics'[breakout_score]))
        ),
        'dimensional dim_artist'[name]
    )
```

```
Avg Breakout Score = AVERAGE('dimensional fact_artist_daily_metrics'[breakout_score])

Avg Growth Velocity = AVERAGE('dimensional fact_artist_daily_metrics'[growth_velocity])

Total Listeners = SUM('dimensional fact_artist_daily_metrics'[lastfm_listeners])
```

Bezpieczeństwo: role użytkowników

Zarządzaj rolami zabezpieczeń

Utwórz nowe role zabezpieczeń i użyj filtrów, aby zdefiniować ograniczenia danych na poziomie wiersza.

Role

+ Nowy

A&R GB

A&R Global

A&R KR

A&R US

Tabele

dimensional br...

dimensional br...

dimensional di...

dimensional di...

dimensional di...

dimensional di...

dimensional di...

dimensional di...

dimensional fa...

Reguły

+ Nowy

Zaznacz wszystko

Usuń

Grupuj

Rozgrupuj

Pokaż dane jeśli

Wszystkie

z tych reguł są prawdziwe

Kolumna	Warunek	Wartość
☐ country	Równa Się	GB

+ Nowy

Przełącz do edytora języka DAX

Ciągłość aktualizacji — Airflow DAG

Petny pipeline jest orkiestrowany przez Airflow 3.0 z harmonogramem schedule="@daily".

Zadania ekstrakcji wykonują się równolegle (nie mają wzajemnych zależności), a transformacja dbt jest uruchamiana dopiero po zakończeniu wszystkich trzech — gwarantuje to, że dbt widzi spójny snapshot wszystkich źródeł.

4. Kontrola jakości

Testy dbt (schema.yml):

Tabela	Kolumna	Test
dim_date	date_id	unique + not_null
dim_date	full_date	unique + not_null
dim_artist	artist_id	unique + not_null
dim_artist	name	not_null

dim_album	album_id	unique + not_null
dim_album	spotify_album_id	unique + not_null
dim_genre	genre_id	unique + not_null
dim_genre	genre_name	unique + not_null
dim_community_tag	tag_id	unique + not_null
dim_community_tag	tag_name	unique + not_null
fact_artist_daily_metrics	artist_id	not_null
fact_artist_daily_metrics	date_id	not_null

Każde uruchomienie pipeline'u kończy się polem dbt test, które automatycznie weryfikuje powyższe ograniczenia. Niespełnienie któregośkolwiek z testów skutkuje niepowodzeniem zadania w Airflow i powiadomieniem operatora.

Podsumowanie

Zgodnie z instrukcją do etapów 3 i 4 wybraliśmy wersję rozszerzoną. Stos technologiczny obejmuje Pandas / SQL / Airflow / dlt / dbt, a model jest fizycznie zaimplementowany jako wielowymiarowa baza danych w architekturze ROLAP (PostgreSQL, schemat dimensional). Cała infrastruktura jest skonteneryzowana (Docker Compose: postgres + pgAdmin + Airflow).

Kluczowe decyzje techniczne i ich uzasadnienie

- ROLAP zamiast MOLAP — PostgreSQL oferuje natywne funkcje okienkowe (LAG, PERCENT_RANK), które wykorzystujemy do obliczenia miar growth_velocity i breakout_score na poziomie warstwy semantycznej. Pozwala to utrzymać cały stos w jednym, otwartym ekosystemie.
- dlt (data load tool) do warstwy staging — natywna obsługa write dispositions (merge / replace), schematu typu i metadanych ładowania pozwala na deklaratywne wdrożenie ekstrakcji przyrostowej.
- dbt do transformacji — wersjonowanie modeli SQL w Git, testy schema.yml (unique, not_null) i ścisłe rozdzielenie warstw (sources -> staging -> dimensional).
- Airflow z DAG-em music_pipeline — harmonogram @daily zapewnia ciągłość aktualizacji.

Wynik: model gwiazdy

Diagram poniżej przedstawia ostateczną postać modelu wielowymiarowego MusicaBI. Centralna tabela faktów (FACT_ARTIST_DAILY_METRICS) jest otoczona trzema głównymi tabelami wymiarów (DIM_ARTIST, DIM_DATE, DIM_ALBUM), a dodatkowo zachowuje cechy schematu płatka śniegu poprzez tabele słownikowe DIM_GENRE i DIM_COMMUNITY_TAG połączone z DIM_ARTIST relacjami M:N za pośrednictwem tabel mostkowych.

